

Katholieke
Universiteit
Leuven

**Faculty of
Engineering Technology
Ghent**

GEAVANCEERDE COMPUTERARCHITECTUUR

Labo 2: Race Conditions

Tom Thys

Contents

1 Introduction **2**

2 Code **2**
2.1 CPU 2
2.2 GPU Atomic 2
2.3 GPU Reduction 2

3 Results **2**
3.1 Performance Comparison 2
3.2 Correctness 3
3.3 Execution Time Comparison 3

A CUDA Source Code: race.cu **4**

1 Introduction

In this lab, we explored the concept of race conditions in GPU programming using CUDA. A race condition occurs when multiple threads access and modify shared data concurrently without proper synchronization, leading to unpredictable results. Through practical exercises, we examined how race conditions can be avoided by using atomic operations and reduction techniques. This allowed us to compare different approaches for computing the maximum value in an array and to assess their performance.

2 Code

2.1 CPU

For the CPU implementation, we used a single-threaded approach to determine the maximum value in the array. The algorithm sequentially iterates through each element, comparing it with the current maximum and updating the value when a larger element is found. This straightforward method avoids any race conditions since only one thread accesses the data at a time, but it does not benefit from parallel execution.

2.2 GPU Atomic

For the GPU implementation, we used an atomic operation to ensure safe access to a shared variable while multiple threads were running in parallel. Each thread compared its assigned element with the current maximum value and updated it using the `atomicMax()` function. This guarantees that only one thread modifies the shared variable at a time, preventing race conditions. Although this approach ensures correctness, it can introduce performance overhead due to thread serialization when many threads attempt to update the same value simultaneously.

2.3 GPU Reduction

In this implementation, we used a reduction algorithm to find the maximum value in parallel. Each thread first compared and stored local maxima within shared memory, followed by iterative reduction steps where the number of active threads was halved in each iteration. Synchronization between threads was required after each step to prevent race conditions. This approach significantly reduces the number of global memory accesses and improves performance compared to the atomic version, especially for larger input sizes.

We used `extern __shared__ int sdata[]` to dynamically allocate shared memory for storing intermediate results. This allows all threads within a block to efficiently access and update the same data without repeatedly reading from slower global memory, which is especially useful when data is frequently shared between threads. However, in this specific implementation, the advantage is limited because shared memory is reallocated for each block and the final result still relies on `atomicMax()`, introducing additional synchronization overhead.

However, once a thread block finishes its local reduction, we use `atomicMax()` to safely combine the results from all blocks. This ensures correctness but can become a bottleneck when many blocks attempt to update the same global variable simultaneously. An alternative approach is to write each block's result to a global array and perform a second reduction step, which can be faster for a large number of blocks.

3 Results

We compared the execution times of the CPU, GPU Atomic, and GPU Reduction implementations for different input sizes. Each measurement was averaged over multiple runs to reduce noise from system variability, this is mainly for the GPU where for 1 run we saw an overhead of initiation.

3.1 Performance Comparison

The CPU implementation showed linear scaling with input size, as expected from its sequential nature. The GPU Atomic version demonstrated significant speed-up for medium-sized inputs but suffered from increased contention as more threads attempted to update the shared maximum simultaneously. We expected the GPU reduction method to outperform the GPU atomic. But because we used extern shared memory in each block we created a lot of overhead, which in turn decreased performance.

3.2 Correctness

All three implementations produced identical results for all tested input sizes, confirming that both GPU approaches correctly handled synchronization and avoided race conditions.

3.3 Execution Time Comparison

Array Size	**CPU (ms)**	**GPU Atomic (ms)**	**GPU Reduction (ms)**
2^{10}	10.55	28.62	41.5
2^{15}	148.79	87.85	121.03
2^{16}	296.71	128.171	164.25
2^{17}	584.52	332.98	317.65
2^{18}	1183.52	337.46	642.613
2^{19}	2364.7	693.5	1276.72
2^{20}	4729.19	1292.84	2522.83

Table 1: Execution time comparison between CPU, GPU Atomic, and GPU Reduction for 10000 loops.

A CUDA Source Code: race.cu

Listing 1: race.cu

```
#include <iostream>
#include <cuda_runtime.h> // This produces an error but works just fine
#include <chrono>

// CUDA find max by checking out with current array item and setting out_max as the highest of both
__global__ void atomic_find_max(int *a, int *out_max, int n) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < n) {
        atomicMax(out_max, a[idx]);
    }
}

__global__ void reduction_find_max(int *a, int *out_max, int n) {
    extern __shared__ int sdata[];
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    int tid = threadIdx.x;

    // Load data into shared memory
    if (idx < n) {
        sdata[tid] = a[idx];
    } else {
        sdata[tid] = -1; // Initialize to a value lower than any possible array element
    }
    __syncthreads();

    // Perform parallel reduction to find max in shared memory
    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            if (sdata[tid + s] > sdata[tid]) {
                sdata[tid] = sdata[tid + s];
            }
        }
        __syncthreads();
    }

    // Write the result for this block to global memory
    if (tid == 0) {
        atomicMax(out_max, sdata[0]);
    }
}

int main() {
    const int N = 1 << 20;
    const int loops = 1000;
    const int threads_per_block = 512;
    const int blocks = (N + threads_per_block - 1) / threads_per_block;
    int a[N], gpu_atomic, gpu_reduction;
    int *data_array, *out_max;
    int init_value = -1; // Initialize to a value lower than any possible array element

    // Initialize host arrays
    for (int i = 0; i < N; i++) {
        a[i] = i;
    }


    // CPU LOOP
    const auto start = std::chrono::steady_clock::now();
    int max_cpu = -1;
    for (int j = 0; j < loops; j++) {
        for (int i = 0; i < N; i++) {
            if (a[i] > max_cpu) {
                max_cpu = a[i];
            }
        }
    }
    const auto end = std::chrono::steady_clock::now();
    const std::chrono::duration<double> elapsed_seconds{end-start};
```

```
69
70      std::cout << "CPU: " << elapsed_seconds.count() << " found: " << max_cpu << std::endl;
71
72
73
74
75      // GPU ATOMIC
76      cudaEvent_t start_cuda, stop_cuda;
77      cudaEventCreate(&start_cuda);
78      cudaEventCreate(&stop_cuda);
79      cudaEventRecord(start_cuda);
80
81      // Allocate device memory
82      cudaMalloc((void**)&data_array, N * sizeof(int));
83      cudaMalloc((void**)&out_max, sizeof(int));
84
85      // Copy data from host to device
86      cudaMemcpy(data_array, a, N * sizeof(int), cudaMemcpyHostToDevice);
87      cudaMemcpy(out_max, &init_value, sizeof(int), cudaMemcpyHostToDevice);
88
89      // Launch kernel and loop for time consistency
90      for (int j = 0; j < loops; j++) {
91          atomic_find_max<<<blocks, threads_per_block>>>(data_array, out_max, N);
92      }
93
94      // Copy result back to host
95      cudaMemcpy(&gpu_atomic, out_max, sizeof(int), cudaMemcpyDeviceToHost);
96
97      cudaDeviceSynchronize();
98      // Free device memory
99      cudaFree(data_array);
100     cudaFree(out_max);
101
102     cudaEventRecord(stop_cuda);
103     cudaEventSynchronize(stop_cuda);
104     float ms;
105     cudaEventElapsedTime(&ms, start_cuda, stop_cuda);
106     cudaEventDestroy(start_cuda);
107     cudaEventDestroy(stop_cuda);
108
109     std::cout << "GPU atomic: ";
110     std::cout << ms << " found: " << gpu_atomic << std::endl;
111
112     // GPU PARALLEL
113     cudaEvent_t start_cuda2, stop_cuda2;
114     cudaEventCreate(&start_cuda2);
115     cudaEventCreate(&stop_cuda2);
116     cudaEventRecord(start_cuda2);
117
118     // Allocate device memory
119     cudaMalloc((void**)&data_array, N * sizeof(int));
120     cudaMalloc((void**)&out_max, sizeof(int));
121
122     // Copy data from host to device
123     cudaMemcpy(data_array, a, N * sizeof(int), cudaMemcpyHostToDevice);
124     cudaMemcpy(out_max, &init_value, sizeof(int), cudaMemcpyHostToDevice);
125
126     // Launch kernel and loop for time consistency
127     for (int j = 0; j < loops; j++) {
128         reduction_find_max<<<blocks, threads_per_block, threads_per_block * sizeof(int)>>>(
                data_array, out_max, N);
129     }
130
131     // Copy result back to host
132     cudaMemcpy(&gpu_reduction, out_max, sizeof(int), cudaMemcpyDeviceToHost);
133
134     cudaDeviceSynchronize();
135     // Free device memory
136     cudaFree(data_array);
137     cudaFree(out_max);
138
139     cudaEventRecord(stop_cuda2);
140     cudaEventSynchronize(stop_cuda2);
```

```
141     cudaEventElapsedTime(&ms, start_cuda2, stop_cuda2);
142     cudaEventDestroy(start_cuda2);
143     cudaEventDestroy(stop_cuda2);
144 
145     std::cout << "GPU reduction: ";
146     std::cout << ms << " found: " << gpu_reduction << std::endl;
147 
148     return 0;
149 }
```